

newdb 源码解析（数据结构 / 算法 / 性能）

newdb/intro

2026 年 5 月 4 日

目录

1 整体架构与数据流	5
1.1 端到端处理流程图	5
1.2 核心对象关系	5
1.3 关键数据结构总览（字段语义 / 不变量 / 生命周期）	6
1.3.1 Row: 逻辑行（内存表示）	6
1.3.2 HeapRowFinger: 物理定位（懒加载的核心索引单元）	6
1.3.3 RecordMetadata: MVCC 元数据（与 slot 对齐存放）	7
1.3.4 TableSchema: 会话级 schema（类型与比较语义的来源）	7
1.3.5 HeapTable: 逻辑表（双形态 + 多索引 + 可见性过滤）	8
1.3.6 WAL 相关: WalRecordHeader / QueuedWalOp	8
1.3.7 WalDecodedRecord: WAL 解码后的“统一记录视图”（用于读取/调试）	8
1.3.8 WalRecoveryStats: 恢复过程的可观测性指标（性能/正确性排查入口）	9
1.3.9 WalTxn: 事务 RAII（保证异常路径也能 rollback）	10
1.3.10 MVCCSnapshot: 读视图的载体（被 HeapTable 消费的关键对象）	10
1.3.11 HeapFileReadView: 只读堆文件视图（mmap/stdio 抽象）	11
1.3.12 HeapLoadOptions: 装载策略开关	12
1.3.13 heap_page::RecordSlice: 页内记录切片（无需拷贝）	12
1.3.14 Status: 显式错误载体（跨层传播的“最小公约数”）	12
1.3.15 Session: 会话对象（线程安全边界明确）	13
1.3.16 Catalog: 数据库目录的文件系统视图	13
1.4 读路径（加载与查询）	13
1.5 写路径（heap 与 WAL）	13
1.6 重构后新增能力（架构视角）	14
2 Heap Page: 页结构、校验与记录切片遍历	14
2.1 关键代码段: 追加记录（slot 表 + free space）	14
2.2 页内布局（逻辑视图）	15
2.3 CRC32 校验	15
2.4 记录遍历: 为什么要对 slot 排序	15
2.5 关键代码段: walk_record_slices 的“排序 + 差分求长度”	15

3	Heap 文件装载：指纹合并、懒加载与物化	16
3.1	关键代码段：合并一页到 latest fingers（逻辑）	16
3.2	目标：把“追加式历史”变成“最新版本视图”	17
3.3	合并规则（merge one page）	17
3.4	懒加载（lazy_decode）	17
3.5	关键代码段：HeapFileReadView 的 mmap/stdio 双模式	18
3.6	物化（finalize / materialize）	18
3.7	复杂度与局部性	18
4	HeapTable：索引、排序缓存与懒解码	18
4.1	关键代码段：decode_heap_slot 的缓存与解码	19
4.2	两种内部形态	19
4.3	可见性过滤（MVCC + tombstone）	19
4.4	索引结构	19
4.5	sorted_indices：排序缓存与“解码一次”优化	20
4.6	关键代码段：sorted_indices 的“排序 key 缓存”	20
4.7	find_by_id：热路径行为	20
5	MVCC：快照可见性与事务水位线	20
5.1	关键代码段：可见性函数 is_visible	21
5.2	可见性判定	21
5.3	MVCCManager：事务集合与 watermarks	21
5.4	关键代码段：begin/commit 与 min_visible_lsn 更新	22
5.5	与 HeapTable 的衔接	22
5.6	实现特征与注意点	22
6	WAL：记录格式、写入线程、分段与恢复	22
6.1	关键代码段：append_record 如何分配 LSN 并计算校验	22
6.2	记录组成：Header + Payload + Checksum	23
6.3	Payload 编码：table + row fields	23
6.3.1	关键代码段：payload 编码（table + row id + row payload）	23
6.4	写入模型：队列 + writer thread	24
6.4.1	关键代码段：enqueue_and_wait（同步等待写入结果）	24
6.5	刷盘策略（durability）	24
6.5.1	关键代码段：Normal 模式的节流刷盘	24
6.6	分段（segment rotate）	25
6.7	恢复（recovery）	25
6.7.1	关键代码段：按事务聚合，commit 时统一 apply	25
6.8	恢复阶段语义：analysis / redo / undo	25
6.9	PITR 与检查点协作	26
7	Session / C API：缓存加载、快照设置与输出约束	26
7.1	关键代码段：Session 如何确保已装载（ensure_loaded）	26
7.2	关键代码段：reload 时的 schema sidecar + lazy 开关	26
7.3	Session：cache_valid 与按需重载	27

7.4	快照设置：影响索引与排序	27
7.5	关键代码段：C API 通过读取日志 tail 获取输出	27
7.6	错误格式：结构化单行输出	28
7.7	C API：以日志尾部作为输出通道	28
8	中层解析 I：cli (CLI、命令分发、业务编排)	28
8.1	模块定位	28
8.2	关键数据结构（中层）	28
8.2.1	ShellState（会话上下文聚合）	28
8.2.2	DemoCliInvocation（一次 CLI 调用的完整语义）	29
8.2.3	WhereQueryContext（查询缓存与策略状态）	29
8.3	核心流程	29
8.3.1	A. 入口主流程（demo_main）	29
8.3.2	B. 命令处理主链（process_command_line）	30
8.4	事务与回退能力（重构后现状）	30
8.5	算法与复杂度（中层视角）	30
8.6	本章复评结论（现状 / 风险 / 建议）	30
8.7	近期修复要点：PAGE id 排序语义一致性	31
9	中层解析 II：engine/include（公共 API 与契约层）	31
9.1	模块定位	31
9.2	头文件分层（建议理解顺序）	31
9.3	关键契约示例	31
9.3.1	Status：统一错误传播约定	31
9.3.2	Session：并发边界写在接口注释里	32
9.3.3	WAL 管理接口：功能完整但实现可替换	32
9.4	本章复评结论（现状 / 风险 / 建议）	32
10	中层解析 III：tests（验证策略与质量护栏）	32
10.1	测试分层概览	32
10.2	代表性测试样例	33
10.2.1	A. WHERE 语义与策略（test_demo_where.cpp）	33
10.2.2	B. 页 IO 容错与懒加载（test_page_io.cpp）	33
10.2.3	C. WAL 并发正确性（test_wal_concurrency.cpp）	33
10.2.4	D. PAGE id 排序回归（test_demo_cli.cpp）	34
10.3	本章复评结论（现状 / 风险 / 建议）	34
11	中层解析 IV：tools（基准与烟测工具链）	34
11.1	工具集定位	34
11.2	关键结构与流程	35
11.2.1	A. newdb_perf：脚本编排器	35
11.2.2	B. newdb_concurrent_perf：并发 WAL 压测	35
11.2.3	C. newdb_smoke：最小可用性验证	35
11.3	本章复评结论（现状 / 风险 / 建议）	36

12 中层解析 V: rust_gui (前端 GUI 独立文档与复评)	36
12.1 文档定位与范围	36
12.2 架构分层复盘	36
12.2.1 A. 交互与状态层 (Vue)	36
12.2.2 B. 原生桥接层 (Tauri / Rust)	36
12.2.3 C. 资源与交付层 (脚本)	36
12.3 近期关键变更归档 (本轮)	37
12.4 事务栈能力 (Undo/Redo)	37
12.5 运维与验证闭环	37
12.6 本章复评结论 (现状 / 风险 / 建议)	37
12.6.1 结论 1: 结构可维护性明显提升	37
12.6.2 结论 2: 用户感知问题闭环较完整	37
12.6.3 结论 3: 仍有可优化空间 (非阻塞)	38
12.7 建议的后续演进顺序	38
A 附录: 与 LevelDB / InnoDB 底层结构对比	38
A.1 一图总览 (结构定位)	38
A.2 1) 存储组织 (Data Layout)	38
A.2.1 newdb	38
A.2.2 LevelDB	38
A.2.3 InnoDB	39
A.3 2) 索引与查询路径	39
A.4 3) WAL / Redo 与恢复模型	39
A.4.1 newdb	39
A.4.2 LevelDB	39
A.4.3 InnoDB	39
A.5 4) 并发控制与 MVCC 深度	39
A.6 5) 空间回收与长期运行形态	39
A.7 6) 写放大 / 读放大 (工程后果)	40
A.8 7) 对 newdb 的演进启发 (可落地)	40
A.9 8) 结论 (结构层面)	40
A.10 9) 优劣评价 (工程决策视角)	40
A.10.1 newdb	40
A.10.2 LevelDB	41
A.10.3 InnoDB	41
A.10.4 按场景选型建议	41
A.11 10) 纳入中层 (cli/engine-include/tests/tools) 后的再评估	41
A.11.1 A. 架构完整性: 从 “内核原型” 提升为 “可用小系统”	41
A.11.2 B. 与 LevelDB/InnoDB 的差距重述 (中层视角)	42
A.11.3 C. 工程风险再分级 (加入中层后)	42
A.11.4 D. 选型建议修订 (纳入中层能力)	42
A.11.5 E. 下一阶段优先级 (基于中层现状)	42
A.12 Executive Summary (1 页决策速览, 面向评审会)	42
A.13 11) 成熟度对照 (已补齐 / 待补齐)	43

1 整体架构与数据流

本章节给出一个从“字节落盘”到“可查询表”的全链路视图，帮助后续章节把局部实现放回整体语境。

1.1 端到端处理流程图

图 1 概括整个项目从交互入口到引擎存储的主数据流（细部分支可在后续章节展开）。依赖方向与仓库约定一致：cli/app → cli/shell → cli/modules → engine/include/newdb（见 docs/MODULE_BOUNDARIES.md、docs/NEWDB_DESIGN_POINT_TO_FILE_MAP.md）。



图 1: newdb 端到端处理流程（主路径概览）

读路径多在 Phase2 → where + sidecar → 惰性解码或索引访问 HeapTable。写路径经 TxnCoordinator → WAL 追加 → 堆文件持久化，并触发 sidecar/缓存失效策略。

1.2 核心对象关系

- **Heap Page**: 固定大小页，保存多条 record 的 payload，并带 CRC32 校验。
- **Heap File (.bin)**: 由一系列 Heap Page 组成；写入时追加 record 到最后一页，满页则新增页。
- **Row**: 逻辑行 (id + attrs)，payload 编码为一组 key=value tuple。
- **HeapRowFinger**: 行的物理定位 (页号 + 页内偏移 + payload 长度)，用于懒加载/快速定位。
- **RecordMetadata**: 从 Row 的内部字段解析出的 MVCC 元数据 (created/deleted lsn, txn id)。

- **HeapTable**: 内存态表, 既可承载物化 rows, 也可承载懒加载视图 + fingers。
- **WAL**: 按 LSN 追加的二进制日志, 支持 commit/rollback/checkpoint 控制记录与恢复回放。
- **Session / C API**: 对外会话层, 负责 schema sidecar 装载、heap 缓存、快照设置与命令执行封装。

1.3 关键数据结构总览 (字段语义 / 不变量 / 生命周期)

本小节集中回答“每个结构体到底有什么字段、字段代表什么、哪些约束必须满足”, 避免在后续章节里碎片化理解。

1.3.1 Row: 逻辑行 (内存表示)

定义见 newdb/engine/include/newdb/row.h:

Listing 1: Row (原型)

```
struct Row {
    int id{0};
    std::unordered_map<std::string, std::string> attrs;
    std::vector<std::string> values;
};
```

字段解释与不变量:

- **id**: 逻辑主键的默认选择。大量路径把 `id==0` 视作“无效/未初始化”(例如解码失败或空 row), 因此业务侧应避免使用 0 作为合法 id。
- **attrs**: 稀疏的键值属性表 (`string->string`)。既承载用户字段, 也承载系统内部字段 (例如 `__deleted`、`__mvcc_*`)。
- **values**: 按 schema 的列序缓存 (“列式缓存”)。它不是存储真源: 由 `HeapTable::rebuild_indexes(schema)` 从 `attrs` 填充, 用于排序/投影时减少 map 查找与字符串构造。

性能含义:

- `attrs` 的哈希表适合稀疏字段, 但频繁访问某个列会产生重复 hash 查找;
- `values` 把热点列访问降为数组下标, 但需要维护 `attr_index` 与 schema 一致性 (见 `HeapTable` 章节)。

1.3.2 HeapRowFinger: 物理定位 (懒加载的核心索引单元)

定义见 newdb/engine/include/newdb/heap_storage.h:

Listing 2: HeapRowFinger (原型)

```
struct HeapRowFinger {
    std::uint64_t page_no{};
    std::uint32_t byte_off_in_page{};
    std::uint32_t payload_len{};
};
```

语义与约束：

- **page_no**: 页号（从 0 开始），必须满足 $\text{page_no} < \text{num_pages}$ 。
- **byte_off_in_page**: 页内 payload 起始偏移。
- **payload_len**: payload 长度。
- **边界约束**: 必须满足 $\text{byte_off_in_page} + \text{payload_len} \leq \text{page_size}$ ，否则解码会越界。

它相当于“堆文件里某一条 record 的物理地址”。`page_io::load_heap_file` 扫描文件时会对同一个 `row.id` 不断覆盖 `finger`，从而形成“最新版本视图”。

1.3.3 RecordMetadata: MVCC 元数据（与 slot 对齐存放）

定义见 `newdb/engine/include/newdb/mvcc.h`:

Listing 3: RecordMetadata（原型）

```
struct RecordMetadata {
    uint64_t created_lsn = 0;
    uint64_t deleted_lsn = 0;
    uint64_t txn_id = 0;
    bool is_tombstone = false;
};
```

字段解释：

- **created_lsn**: 该版本创建时的逻辑时间（与 snapshot lsn 比较）。
- **deleted_lsn**: 删除生效的逻辑时间；0 表示未删除。
- **txn_id**: 创建该版本的事务 id；用于“读不见未提交写”。
- **is_tombstone**: 是否墓碑行；通常由 `__deleted=1` 推导。

生命周期与对齐方式：

- 物化形态: `HeapTable::row_meta[i]` 与 `rows[i]` 对齐；
- 懒加载形态: `HeapTable::row_meta[slot]` 与 `heap_fingers_[slot]` 对齐。

1.3.4 TableSchema: 会话级 schema（类型与比较语义的来源）

定义见 `newdb/engine/include/newdb/schema.h`:

Listing 4: TableSchema（核心字段节选）

```
struct TableSchema {
    std::string table_label;
    std::vector<AttrMeta> attrs;
    std::string primary_key{"id"};
    std::uint32_t heap_format_version{0};
};
```

关键语义：

- **attrs**: 声明型字段列表（名字 + 类型）。注意：存储层允许出现“schema 未声明的额外 attrs”（`validate_storage_row` 只校验已声明字段能否解析）。
- **primary_key**: 主键字段名；若无效会回退到 `id`。
- **compare_attr**: 排序/谓词比较的权威入口。对 `int/double/bool` 会尝试数值化比较，失败则回退到字符串比较。

1.3.5 HeapTable: 逻辑表（双形态 + 多索引 + 可见性过滤）

定义见 `newdb/engine/include/newdb/heap_table.h`。最关键的是它同时承载两种表示：

- **物化**: `rows` 非空，索引值指向 `rows` 下标；
- **懒加载**: `heap_file_ != nullptr` 且 `rows` 为空，索引值指向 `heap_fingers_ slot`，需要 `decode_heap_slot` 才能得到 `Row`。

因此对外“看起来都是表”，对内“既可能是内存表也可能是磁盘表视图”。这一点直接决定了很多函数的分支与性能特征（详见 `HeapTable` 章节）。

1.3.6 WAL 相关: WalRecordHeader / QueuedWalOp

`WalRecordHeader` 定义见 `newdb/engine/include/newdb/wal_manager.h`:

Listing 5: `WalRecordHeader`（原型）

```
struct WalRecordHeader {
    uint32_t magic = 0x57414C30; // "WALO"
    uint64_t lsn = 0;
    uint64_t txn_id = 0;
    uint32_t payload_len = 0;
    uint16_t checksum = 0;
    uint8_t type = 0; // WalOp
    uint8_t flags = 0;
} __attribute__((packed));
```

注意点：

- **packed**: 为确保 on-disk layout 稳定，header 被打包；这要求读写时严格按该结构体大小序列化。
- **checksum 的覆盖范围**: 实现里会跳过 `checksum` 字段本身，覆盖 header 其余部分与 `payload`。
- **type/flags**: `type` 存 `WalOp`，`flags` 预留（方便未来扩展）。

`QueuedWalOp` 是 `WalManager` 的内部队列单元（同一头 + `payload` + `flush` 标记 + `promise`）：它决定了 WAL 写入天然串行，并能把 `writer` 线程错误同步回传给调用方（见 `WAL` 章节）。

1.3.7 WalDecodedRecord: WAL 解码后的“统一记录视图”（用于读取/调试）

定义见 `newdb/engine/include/newdb/wal_manager.h`:

Listing 6: `WalDecodedRecord`（原型）

```
struct WalDecodedRecord {
    uint64_t lsn{0};
```



```
uint64_t txn_id{0};
WalOp op{WalOp::CHECKPOINT};
std::string table;
Row row;
bool has_row{false};
};
```

生命周期（在哪填充/消费）：

- **填充**：WalManager::read_all_records(schema, out) 遍历所有 WAL segment，逐条读取 header+payload、校验 checksum、解码 table/row 字段，构造 WalDecodedRecord push 到 out。
- **消费**：主要用于“把 WAL 当成可读流”输出/调试/回放检查；它不是恢复主路径的中间表示（恢复主路径用 PendingTxnOps 聚合）。

设计要点：

- **has_row** 把控制记录与行记录区分开：COMMIT/ROLLBACK/CHECKPOINT 等控制记录不带 row payload。
- **row 的 schema 语义**：row 的 payload 解码不强依赖 schema (tuple codec 是 key=value)，但某些上层展示/校验可能用 schema 做类型解释。

1.3.8 WalRecoveryStats：恢复过程的可观测性指标（性能/正确性排查入口）

定义见 newdb/engine/include/newdb/wal_manager.h：

Listing 7: WalRecoveryStats（字段节选）

```
struct WalRecoveryStats {
    std::uint64_t records_read{0};
    std::uint64_t checksum_failures{0};
    std::uint64_t decode_failures{0};
    std::uint64_t apply_count{0};
    std::uint64_t scanned_segments{0};
    std::uint64_t skipped_segments{0};
    std::uint64_t indexed_records{0};
    std::uint64_t indexed_segments{0};
    std::uint64_t indexed_offsets{0};
    std::uint64_t seek_skipped_records{0};
    std::uint64_t begin_ms{0};
    std::uint64_t end_ms{0};
    std::uint64_t elapsed_ms() const { ... }
};
```

生命周期（在哪填充/消费）：

- **填充**：WalManager::recover(out_table, schema, stats) 在恢复全流程中持续更新统计值：
 - 索引阶段：为 segment 构建稀疏 offset 索引时更新 indexed_*；
 - 扫描阶段：逐段读取记录时更新 records_read/scanned_segments；
 - 异常与容错：checksum/解码失败分别累加，并可能导致提前失败返回；

- apply 阶段：提交事务时更新 `apply_count`。
- **消费**：调用方可通过 `last_recovery_stats()` 取到最近一次恢复的 stats，用于判断“恢复慢在哪 / 哪类失败最多 / offset seek 是否生效”。

优化与诊断价值：

- **records_read vs apply_count**：若差距巨大，可能意味着大量控制记录/回滚事务/被过滤 table 的记录；
- **indexed_offsets / seek_skipped_records**：用于评估 offset seek 的收益（尤其是设置了 `NEWDB_RECOVER_MIN_LSN` 时）。

1.3.9 WalTxn：事务 RAII（保证异常路径也能 rollback）

定义见 `newdb/engine/include/newdb/wal_manager.h`：

Listing 8: WalTxn（原型）

```
class WalTxn {
public:
    WalTxn(WalManager& wal, uint64_t txn_id);
    ~WalTxn();
    void commit();
    void rollback();
    uint64_t txn_id() const;
private:
    WalManager* wal_;
    uint64_t txn_id_;
    bool active_;
};
```

生命周期（在哪填充/消费）：

- **创建**：业务侧在开始事务时构造 WalTxn（内部会调用 `wal.begin_transaction(txn_id)`）。
- **提交**：调用 `commit()` 追加 COMMIT 记录并令 `active_=false`。
- **异常路径**：若对象析构时仍 `active_==true`，析构函数会自动 rollback（追加 ROLLBACK 记录）。

不变量与误用风险：

- **单次提交/回滚**：`commit/rollback` 之后 `active_` 变 `false`，应避免二次调用造成重复控制记录。
- **作用域即事务边界**：这要求调用方把 WalTxn 放在“事务应结束”的同一作用域内，避免长生命周期导致锁/资源长期占用（取决于上层实现）。

1.3.10 MVCCSnapshot：读视图的载体（被 HeapTable 消费的关键对象）

定义见 `newdb/engine/include/newdb/mvcc.h`：

Listing 9: MVCCSnapshot (原型)

```
struct MVCCSnapshot {
    uint64_t snapshot_lsn = 0;
    uint64_t min_visible_lsn = 0;
    std::unordered_set<uint64_t> active_txns;
    bool is_visible(uint64_t created_lsn, uint64_t deleted_lsn, uint64_t creator_txn) const;
    bool is_visible(const RecordMetadata& meta) const;
};
```

生命周期（在哪填充/消费）：

- **填充**：MVCCManager::create_snapshot() 持锁复制 active_txns_，并用 wal.current_lsn() 作为 snapshot_lsn（再带上 min_visible_lsn 水位线）。
- **消费**：
 - Session::set_snapshot(snapshot) 把快照写入 HeapTable::active_snapshot;
 - HeapTable::is_row_visible(slot,row) 读取 row_meta[slot] 并调用 snapshot.is_visible(meta);
 - 因此“建索引/排序/查找”等读路径都会受到该快照的过滤影响。
- **销毁**：快照是值对象（包含一个 unordered_set 拷贝），通常随 session/调用栈生命周期结束自动释放。

性能含义：

- **active_txns 的拷贝成本**：创建快照是 $O(|active_txns|)$ 的（复制集合）。若未来活跃事务数大，可能需要更紧凑表示（例如 epoch/位图/范围）或共享结构。
- **可见性检查的热度**：is_row_visible 会在建索引、排序、扫描中被大量调用，这要求 row_meta 与 slot 对齐且访问尽量 cache-friendly。

1.3.11 HeapFileReadView：只读堆文件视图（mmap/stdio 抽象）

定义见 newdb/engine/include/newdb/heap_file_read_view.h：

Listing 10: HeapFileReadView（接口节选）

```
class HeapFileReadView {
public:
    static std::shared_ptr<HeapFileReadView> try_open(const char* path);
    std::size_t page_size() const;
    std::size_t num_pages() const;
    const unsigned char* page_data(std::size_t page_no) const; // nullptr if not mmap-backed
    bool read_page_copy(std::size_t page_no, unsigned char* buf) const; // always works in range
};
```

关键语义：

- **两种后端统一**：如果平台支持且映射成功，page_data 直接返回映射指针；否则返回 nullptr，上层改用 read_page_copy。

- **对 HeapTable 的影响**：懒加载形态下，decode_heap_slot 会优先走 page_data（零拷贝），否则走单页缓存 + read_page_copy（每页一次 I/O）。
- **生命周期**：这是只读视图，析构时会 unmap（若曾映射）；因此持有它的 shared_ptr 必须覆盖整个“懒加载表”的使用期。

1.3.12 HeapLoadOptions：装载策略开关

定义见 newdb/engine/include/newdb/heap_storage.h：

Listing 11: HeapLoadOptions（原型）

```
struct HeapLoadOptions {
    bool lazy_decode{false};
};
```

它把 load_heap_file 的行为切成两种模式：

- **lazy_decode=false（默认）**：物化所有 Row 到 HeapTable::rows；
- **lazy_decode=true**：只构建 finger + 元数据，并用 HeapFileReadView 提供按需解码。

1.3.13 heap_page::RecordSlice：页内记录切片（无需拷贝）

定义见 newdb/engine/include/newdb/heap_page.h：

Listing 12: RecordSlice（原型）

```
struct RecordSlice {
    unsigned slot_index{};
    const unsigned char* payload{nullptr};
    std::size_t payload_length{0};
};
```

它代表“页内某条 record 的视图”，而不是拷贝：

- **payload 指针生命周期**：只在访问该页缓冲（或 mmap 映射）有效期间有效；回调不能把指针保存到页之外长期使用。
- **slot_index**：是 slot 表中的逻辑编号，不保证与 payload 偏移有序；遍历实现会按偏移排序后回调（见 Heap Page 章节）。

1.3.14 Status：显式错误载体（跨层传播的“最小公约数”）

定义见 newdb/engine/include/newdb/error.h：

Listing 13: Status（原型）

```
struct Status {
    bool ok{true};
    std::string message;
    static Status Ok();
    static Status Fail(std::string msg);
    explicit operator bool() const { return ok; }
};
```

设计意图：

- **无异常依赖**：底层 I/O/解码/校验都用 Status 返回，避免异常跨 ABI/线程边界。
- **调用习惯统一**：if (!st.ok) return st; 这种风格贯穿 heap/WAL/session。

1.3.15 Session：会话对象（线程安全边界明确）

定义见 newdb/engine/include/newdb/session.h。它的“关键点”不是字段数量，而是并发语义：

- **mut_ 保护范围**：保护 cache_valid、schema 重载副作用、以及 table 的一致视图。
- **mutable_heap 的危险性**：它只在函数调用期间持锁，返回的 HeapTable* 不能跨并发 reload/reset/invalidate 安全使用。
- **lock_heap 的正确用法**：返回 RAII 的 HeapAccess，持锁直到对象析构，从而保证指针在作用域内安全。

这实际上是“把并发边界写进类型系统/注释契约里”，避免误用。

1.3.16 Catalog：数据库目录的文件系统视图

定义见 newdb/engine/include/newdb/catalog.h：

- **root_**：数据库根目录（可为空则用当前目录回退）；
- **current_db_**：当前选中的数据库名；
- **.dbinfo**：以 sidecar 文件标记“这是一个 db 目录”（见实现）。

它不是存储引擎核心热路径，但决定了“数据文件与工作区”的组织方式，是上层命令/工具正确工作的前提。

1.4 读路径（加载与查询）

1. 读取 .attr sidecar 得到 TableSchema（字段类型、主键、heap 格式版本）。
2. 扫描 .bin：逐页 CRC 校验、逐 record 解码，按 row.id 合并出“最新版本”的 HeapRowFinger。
3. 生成 HeapTable：
 - 懒加载：保留只读视图（可 mmap）+ fingers + 元数据；
 - 物化：一次性解码所有 payload 成 rows，然后构建索引。
4. 查询：走 index_by_id / index_by_pk_value 或排序缓存；在关键路径上应用 MVCC 可见性过滤。

1.5 写路径（heap 与 WAL）

当前代码中存在两条可并行理解的写入机制：

- **直接写 heap 文件**：把 Row 编码后追加到最后一页（满则新页），并更新页校验。
- **写 WAL**：把行操作编码到 WAL payload，按事务聚合，在恢复时回放到 HeapTable。

两者可组合为“WAL 保障持久性 + heap 作为主存储”（是否已完整打通取决于上层 cli/命令层如何调用）。

1.6 重构后新增能力（架构视角）

相较早期“单路径回放”形态，当前工程在事务与恢复面上已补齐若干关键能力：

- **恢复闭环**：WAL 从单一回放扩展为 analysis/redo/undo 三阶段恢复；
- **任意回退**：命令层支持 savepoint 粒度回退/重做与事务级回滚；
- **时间点恢复**：支持按 LSN/时间戳约束回放终点（PITR）；
- **GUI 运维化**：Rust GUI 提供事务栈、回看、崩溃矩阵与趋势看板入口，形成“执行-验证-复盘”链路。

2 Heap Page: 页结构、校验与记录切片遍历

对应实现：newdb/engine/src/heap/heap_page_waterfall.cpp

2.1 关键代码段：追加记录（slot 表 + free space）

追加一条记录的核心逻辑是“payload 向前增长 + slot 表向后增长”，并用 header 维护两者之间的空洞：

Listing 14: append_encoded_record 的关键步骤（节选）

```
const std::size_t need = rec_len + sizeof(std::uint16_t);
if (cur_free_size < need) {
    return false;
}

const unsigned short rec_offset = cur_free_space;
std::memcpy(page + rec_offset, encoded, rec_len);

const unsigned short new_num = cur_num + 1;
std::uint16_t* new_slot = slot_ptr(page, new_num);
*new_slot = rec_offset;

const unsigned short slot_region_begin =
    static_cast<unsigned short>(reinterpret_cast<unsigned char*>(new_slot) - page);
const unsigned short new_free_space = static_cast<unsigned short>(rec_offset + rec_len);
const unsigned short new_free_size =
    static_cast<unsigned short>(slot_region_begin - new_free_space);

header->set_num_records(new_num);
header->set_free_space(new_free_space);
header->set_free_size(new_free_size);
```

细节解读：

- **为何 need 要加 2 字节**：每条记录除了 payload，还要在 slot 表写入一个 uint16_t 的偏移值。
- **slot_ptr 的语义**：slot 表位于页尾，slot_ptr(page, n) 返回“n 条记录时，新 slot 应写入的位置”。
- **free_size 的计算**：用“slot 表起始地址”减去“payload 已写到的位置”，保持两端不相撞。

2.2 页内布局（逻辑视图）

Heap Page 使用 Waterfall 的 `page_header_t`，并维护三类区域：

- **Header**：记录页类型、record 数、free space 指针、free size、CRC32 等；
- **Payload 区**：自前向后增长，连续写入每条 record 的 payload（不显式存长度）；
- **Slot 表**：自页尾向前增长，每新增一条记录，写入一个 `uint16_t` 的 payload 偏移。

新增记录时，需要的空间为：

$$\text{need} = \text{payload_len} + \text{sizeof}(\text{uint16_t})$$

若 `free_size < need` 则追加失败，调用方通常会新开一页再试。

2.3 CRC32 校验

页校验由 `page_base::compute_checksum()` 计算，header 保存的 `crc32` 与之相等则通过。装载时校验失败的页会被跳过（容错策略），写入时每次落盘前会更新 CRC32。

2.4 记录遍历：为什么要对 slot 排序

Slot 表中记录的“slot 下标顺序”与 payload 的“物理偏移升序”并不一致（例如新写入的 slot 在更靠近页尾的位置）。而 record 的长度并未显式存储，因此遍历时必须按 payload 偏移升序来恢复每条记录长度：

1. 读取 header 的 record 数 `num` 与 free space 指针 `free_space`；
2. 取到 slot 表全部偏移数组 `all_slots[0..num-1]`；
3. 构造 `order`，按 `all_slots[order[k]]` 升序排序；
4. 第 k 条 record 的起点为 `all_slots[order[k]]`，终点为下一条起点（或 `free_space`），长度为两者差。

该算法的时间复杂度为 $O(num \log num)$ （页内排序），但页内 record 数通常受限于页大小，因此在整体扫描中表现稳定。

2.5 关键代码段：walk_record_slices 的“排序 + 差分求长度”

Listing 15: `walk_record_slices` 的核心思路（节选）

```
// 1) order = [0..num-1]
std::sort(order.begin(), order.end(), [&](unsigned short a, unsigned short b) {
    return all_slots[a] < all_slots[b];
});

// 2) rec_len = next_offset - rec_offset
const unsigned short rec_offset = all_slots[slot_index];
const unsigned short next_offset =
    (k + 1 < num) ? all_slots[order[k + 1]] : free_space;
const std::size_t rec_len = static_cast<std::size_t>(next_offset - rec_offset);
slice.payload = page + rec_offset;
slice.payload_length = rec_len;
```

为什么这个设计很关键：

- **不存长度，靠差分恢复**：页内不为每条记录保存长度，节省空间，但要求遍历时能从“偏移序列”恢复长度。
- **必须排序**：如果直接按 slot 下标遍历，next_offset 不一定比当前 offset 大，差分会变负或越界。
- **边界使用 free_space**：最后一条记录的终点就是 header 维护的 free space 指针。

3 Heap 文件装载：指纹合并、懒加载与物化

对应实现：newdb/engine/src/io/page/page_io.cpp、newdb/engine/src/heap/heap_file_read_view.cpp

3.1 关键代码段：合并一页到 latest fingers（逻辑）

装载的核心是：扫描所有 record，按 row.id 合并出“最新的物理位置 (finger)”，并处理 tombstone 删除。下面是 merge_one_page_into_fingers 的关键逻辑（做了少量重排以便阅读）：

Listing 16: merge_one_page_into_fingers（节选）

```
if (!heap_page::verify_checksum(page_base, psz)) {
    return Status::Ok(); // checksum mismatch: skip page
}

heap_page::walk_record_slices(page_base, psz, [&](const RecordSlice& slice) {
    Row row;
    if (!codec::decode_heap_payload_to_row(slice.payload, slice.payload_length, row)) {
        return true; // skip bad record
    }
    Status vst = validate_storage_row(schema, row);
    if (!vst.ok) { fatal = vst; return false; }

    const RecordMetadata meta = extract_record_metadata(row, ++lsn_counter);
    if (row.attrs["__deleted"] == "1") {
        latest.erase(row.id);
        latest_meta.erase(row.id);
        return true;
    }

    HeapRowFinger finger{};
    finger.page_no = page_no;
    finger.byte_off_in_page = off_in_page;
    finger.payload_len = slice.payload_length;
    latest[row.id] = finger;
    latest_meta[row.id] = meta;
    return true;
});
```

关键点解释：

- **checksum mismatch 的策略**：直接跳过页（容错）；这会导致数据“可能缺行”，但能尽量从坏文件中恢复出可用子集。

- **以 row.id 为合并键**：同 id 的后写入版本会覆盖 latest[id]，形成“最后写入赢”的视图。
- **tombstone 的语义**：遇到 __deleted=1 会删除 latest 中该 id，等价于“最新版本是删除”。
- **LSN 的来源**：若 row 自带 __mvcc_created_lsn 则用它，否则用装载过程的递增计数作为 fallback（保证可比较但不等同于 WAL LSN）。

3.2 目标：把“追加式历史”变成“最新版本视图”

堆文件是追加式写入：同一个 row.id 可能在文件中出现多次（更新/删除后写入新版本或 tombstone）。装载过程的核心任务是扫描所有 record，并为每个 row.id 选择一个“最新”位置：

- latest: row.id → HeapRowFinger
- latest__meta: row.id → RecordMetadata

3.3 合并规则（merge one page）

对每一页：

1. 若页 CRC32 校验失败，跳过该页（非 fatal）；
2. 遍历页内所有 record slice，解码为 Row；
3. 校验存储值与 schema 的可解析性（例如 int/double/bool 等）；
4. 若 __deleted=1：从 latest/latest_meta 移除该 id；
5. 否则写入/覆盖：latest[id] = (page_no, offset, len)，并保存解析出的 MVCC 元数据。

备注：装载过程内部维护一个自增的 lsn_counter 作为 fallback 的 created_lsn（当行内没带 __mvcc_created_lsn 时使用）。

3.4 懒加载（lazy__decode）

若启用懒加载，系统会尝试构造一个只读视图 HeapFileReadView：

- 支持平台上可直接 mmap 整个 heap 文件，page_data(page_no) 返回指向映射内存的指针；
- 在不支持 mmap 或无法映射时，保持路径与页大小信息，按需读用 read_page_copy 读页。

随后 HeapTable 只保存：

- 排好序的 ids（按 page_no 再按 id）
- 与 ids 对齐的 fingers 与 row_meta
- 只读视图指针

查询时按 slot 解码：定位页、取 payload、调用 tuple codec 解码成 Row。

3.5 关键代码段：HeapFileReadView 的 mmap/stdio 双模式

Listing 17: HeapFileReadView::page_data 与 read_page_copy (节选)

```
const unsigned char* HeapFileReadView::page_data(size_t page_no) const {
    if (mapped_ == nullptr || page_no >= num_pages_) return nullptr;
    return static_cast<const unsigned char*>(mapped_) + page_no * psz_;
}

bool HeapFileReadView::read_page_copy(size_t page_no, unsigned char* buf) const {
    if (mapped_ != nullptr) {
        std::memcpy(buf, base + page_no * psz_, psz_);
        return true;
    }
    FILE* fp = std::fopen(path_.c_str(), "rb");
    std::fseek(fp, page_no * psz_, SEEK_SET);
    std::fread(buf, 1, psz_, fp);
    std::fclose(fp);
    return true;
}
```

这意味着懒加载查询路径在“能 mmap 的平台”上很像纯内存访问；在 Windows/不可 mmap 情况下则会退化为按需读页拷贝（可通过 HeapTable 的单页缓存减轻抖动，见下一章）。

3.6 物化 (finalize / materialize)

若不启用懒加载，则会对排序后的 ids 逐个：

1. 按 finger 定位到页；
2. 验证页 CRC；
3. 解码 payload 得到 Row，剥离 MVCC 内部字段；
4. 校验 schema；
5. push 到 HeapTable::rows，并把 RecordMetadata push 到 row_meta；
6. 最后 rebuild_indexes(schema)。

3.7 复杂度与局部性

设总记录数为 N ，去重后的 id 数为 M ：

- 扫描与解码： $O(N)$ （每条 record 至多解码一次）
- 构建并排序 ids： $O(M \log M)$

按 $(page_no, id)$ 排序的目的，是让后续物化时尽量顺序访问同一页，提高缓存命中与 I/O 局部性。

4 HeapTable: 索引、排序缓存与懒解码

对应实现：newdb/engine/src/heap/heap_table.cpp

4.1 关键代码段：decode_heap_slot 的缓存与解码

懒加载形态下的“按 slot 解码”是很多读路径的基础，它做了一个非常关键的优化：**单页缓存**。

Listing 18: decode_heap_slot: mmap 命中或单页缓存（节选）

```
const unsigned char* page_ptr = file->page_data(f.page_no);
if (page_ptr == nullptr) {
    if (heap_cached_page_no_ != f.page_no || heap_cached_page_buf_.size() != psz) {
        heap_cached_page_buf_.resize(psz);
        file->read_page_copy(f.page_no, heap_cached_page_buf_.data());
        heap_cached_page_no_ = f.page_no;
    }
    page_ptr = heap_cached_page_buf_.data();
}
codec::decode_heap_payload_to_row(page_ptr + f.byte_off_in_page, f.payload_len, out);
strip_mvcc_internal_attrs(out);
```

解释：

- **mmap 命中**：page_data 直接给出指针，不产生额外 I/O 与拷贝。
- **mmap 未命中**：通过 read_page_copy 把页读入 heap_cached_page_buf_，并记住 heap_cached_page_no_。
- **为什么单页缓存有效**：很多扫描/排序会局部访问同一页内的多个 slot，单页缓存能把“每行一次读页”降为“每页一次读页”。

4.2 两种内部形态

- **物化形态**：rows 持有所有 Row；查询主要在内存数组 + hash 索引上完成。
- **懒加载形态**：heap_file_ + heap_fingers_ 保存物理定位；访问时解码 slot。

这两种形态共享同一套逻辑接口，但性能特征差异明显：懒加载更省内存，物化更稳定更快。

4.3 可见性过滤 (MVCC + tombstone)

is_row_visible(slot,row) 同时处理：

- tombstone (__deleted=1) 不可见；
- 若未设置 snapshot，则默认可见；
- 若设置 snapshot，则用 RecordMetadata 的 created/deleted/txn id 做可见性判断。

因此任何“构建索引/排序/查找”的结果都必须经过该过滤，才能对外呈现一致视图。

4.4 索引结构

- **index_by_id**：id → slot/index，平均 $O(1)$ ；
- **index_by_pk_value**：主键非 id 时构建 pk_value → slot；
- **attr_index**：字段名到列序号的映射（用于 values 向量快速取值）。

注意：懒加载形态下会根据 schema 的主键策略选择性构建 index_by_pk_value，避免不必要的解码与内存开销。

4.5 sorted_indices: 排序缓存与“解码一次”优化

`sorted_indices(schema, order_key, dir)` 会返回一个 slot/index 列表引用, 并按 `order_key` 排序:

- 结果按 `order_key` 进行缓存 (升序/降序各一份);
- 缓存 key 数量上限为 16, 避免无界增长;
- 懒加载且 `order_key != id` 时, 先为候选 slot 计算并缓存排序 key, 再 sort, 避免比较函数中重复解码导致的高倍放大。

首次计算复杂度:

$$O(N \cdot \text{decode}) + O(N \log N)$$

命中缓存后为 $O(1)$ 。

4.6 关键代码段: sorted_indices 的“排序 key 缓存”

懒加载且 `order_key != id` 时, 为避免 sort 比较器内反复解码, 代码会先构造 slot→key 的映射:

Listing 19: sorted_indices: 先解码一次再排序 (节选)

```
std::unordered_map<size_t, std::string> sort_keys;
for (size_t slot : indices) {
    Row row;
    decode_heap_slot(slot, row);
    auto it = row.attrs.find(order_key);
    sort_keys.emplace(slot, it != row.attrs.end() ? it->second : "");
}
std::sort(indices.begin(), indices.end(), [&](size_t ia, size_t ib) {
    int cmp = schema.compare_attr(order_key, sort_keys[ia], sort_keys[ib]);
    return dir == Asc ? (cmp < 0) : (cmp > 0);
});
```

这把解码成本从“比较器内的 $O(N \log N)$ 次解码”降到了“预处理的 $O(N)$ 次解码”, 在 N 较大时差距非常明显。

4.7 find_by_id: 热路径行为

若 `index_by_id` 命中:

- 物化: 直接返回 `rows[idx]` (额外做可见性检查)
- 懒加载: 解码该 slot 到 scratch row, 并返回其指针 (同样做可见性检查)

若索引未命中且物化形态, 会退化为全表扫描 $O(N)$; 懒加载形态则直接返回 `nullptr` (避免隐式触发大量解码)。

5 MVCC: 快照可见性与事务水位线

对应实现: `newdb/engine/src/mvcc/snapshot/mvcc.cpp` (快照与管理器)、`newdb/engine/src/io/page/page_io.cpp` (元数据提取)、`newdb/engine/src/heap/heap_table.cpp` (可见性应用)

5.1 关键代码段：可见性函数 is_visible

Listing 20: MVCCSnapshot::is_visible 的判定顺序（节选）

```
// Created after snapshot? Not visible
if (created_lsn > snapshot_lsn) return false;

// Deleted before or at snapshot? Not visible
if (deleted_lsn != 0 && deleted_lsn <= snapshot_lsn) return false;

// Creator transaction not yet committed at snapshot time?
if (active_txns.find(creator_txn) != active_txns.end()) {
    return false;
}
return true;
```

解读与推论：

- **判定顺序很重要**：先比较 LSN，再看活跃事务集合，使得“快照之后创建”的版本无条件不可见。
- **deleted_lsn 的闭区间**：使用 $\leq$ 表示“在快照时刻已删除”的版本不可见；这把删除视作在某个 LSN 上生效。
- **active_txns 的语义**：若创建事务仍在 active 集合里，即使 created_lsn 早于 snapshot，也不可见（避免读到未提交写）。

5.2 可见性判定

对一条记录（版本）而言，其元数据可抽象为三元组：

$$(\text{created_lsn}, \text{deleted_lsn}, \text{creator_txn})$$

快照由：

$$(\text{snapshot_lsn}, \text{active_txns})$$

组成。当前实现的可见性规则是：

- 若 $\text{created_lsn} > \text{snapshot_lsn}$ ：不可见（快照后创建）；
- 若 $\text{deleted_lsn} \neq 0 \wedge \text{deleted_lsn} \leq \text{snapshot_lsn}$ ：不可见（快照时已删除）；
- 若 $\text{creator_txn} \in \text{active_txns}$ ：不可见（创建事务未提交）；
- 其他情况：可见。

5.3 MVCCManager：事务集合与 watermarks

MVCCManager 在互斥锁保护下维护：

- **active_txns_**：活跃事务集合；
- **committed_txn_lsn_**：提交事务到 commit lsn 的映射（可用于 purge）；
- **min_visible_lsn_**：简化版本的水位线（当前实现以 max commit lsn 更新）。

创建快照时使用 WAL 的 `current_lsn()` 作为 `snapshot_lsn`，并复制 active 集合。

5.4 关键代码段: begin/commit 与 min_visible_lsn 更新

Listing 21: MVCCManager 的事务状态变迁 (节选)

```
uint64_t begin_transaction() {
    lock_guard lg(mut_);
    uint64_t txn_id = next_txn_id++;
    active_txns_.insert(txn_id);
    return txn_id;
}

void commit_transaction(uint64_t txn_id, uint64_t commit_lsn) {
    lock_guard lg(mut_);
    active_txns_.erase(txn_id);
    committed_txn_lsn_[txn_id] = commit_lsn;
    min_visible_lsn_ = std::max(min_visible_lsn_, commit_lsn);
}
```

这里的 `min_visible_lsn_` 更新是“简化水位线”: 它记录目前见过的最大 commit lsn。如果未来要用它做 GC/截断, 需要改成“系统中所有活跃快照的最小值”一类更严格的定义。

5.5 与 HeapTable 的衔接

Heap 文件装载时会从 Row 内部字段提取 `RecordMetadata`, 并与 slot 对齐存放。HeapTable 在:

- 建索引 (`rebuild_indexes`)
- 排序 (`sorted_indices`)
- 查找 (`find_by_id`)

等路径上调用 `is_row_visible` 做过滤, 从而把 MVCC 变成“对外一致的逻辑视图”。

5.6 实现特征与注意点

- 这套 MVCC 更偏向“快照时刻的读可见性”, 而不是完整的多版本存储引擎 (例如缺少 undo/版本链)。
- `min_visible_lsn` 的更新策略目前是简化版, 若后续要做更严格的 GC/compaction, 需要更精确的全局最小快照 lsn 跟踪。

6 WAL: 记录格式、写入线程、分段与恢复

对应实现: `newdb/engine/src/wal/writer/wal_manager.cpp`、`newdb/engine/src/wal/codec/wal_codec.cpp`

6.1 关键代码段: append_record 如何分配 LSN 并计算校验

Listing 22: `append_record`: 构造 header + payload + checksum (节选)

```
hdr.magic = 0x57414C30;
hdr.lsn = next_lsn_.fetch_add(1) + 1;
```

```
hdr.txn_id = txn_id;
hdr.payload_len = (uint32_t)payload.size();
hdr.type = (uint8_t)op;
hdr.checksum = compute_checksum(&hdr, payload.data(), (uint32_t)payload.size());
```

要点:

- **LSN 分配是并发安全的**: `next_lsn_` 是原子计数器, 多线程调用也能得到全序 LSN。
- **checksum 覆盖范围**: header (除 checksum 字段) + payload。恢复时先读 header/payload, 再验证 checksum, 失败则停止回放并报错。

6.2 记录组成: Header + Payload + Checksum

每条 WAL 记录由固定头与可选 payload 组成:

- **magic**: 常量 0x57414C30, 用于快速识别;
- **lsn**: 单调递增的日志序列号;
- **txn_id**: 事务 id;
- **payload_len**: payload 长度;
- **type**: 操作类型 (INSERT/UPDATE/DELETE/COMMIT/ROLLBACK/CHECKPOINT/...);
- **checksum**: CRC16-CCITT, 覆盖 header(除 checksum 字段) 与 payload。

其中 checksum 计算使用 CCITT 多项式, 作为轻量一致性检查。

6.3 Payload 编码: table + row fields

payload 的二进制布局由 `wal_codec` 定义:

- u16 table_name_len + table name bytes
- 若为行操作 (INSERT/UPDATE/DELETE):
 - u32 row_id
 - u32 row_payload_len
 - row payload bytes (由 tuple codec 编码得到)

6.3.1 关键代码段: payload 编码 (table + row id + row payload)

Listing 23: `walcodec::build_payload` 的布局 (节选)

```
append_u16_le((uint16_t)table.size(), out);
out.insert(out.end(), table.begin(), table.end());
if (row_id != nullptr) {
    append_u32_le(*row_id, out);
    append_u32_le((uint32_t)row_payload->size(), out);
    out.insert(out.end(), row_payload->begin(), row_payload->end());
}
```

这说明 WAL payload 是“轻量自描述”的: 只编码 table name 与可选的 row 字段; row 的内部结构由 heap payload (tuple codec) 负责。

6.4 写入模型：队列 + writer thread

WalManager::open() 会启动 writer 线程：

- 前台线程调用 append_record/flush/commit/rollback;
- 操作被封装为 QueuedWalOp 进入队列;
- writer 线程串行写入文件，并通过 promise/future 把结果返回给调用方。

这样做的好处是：写入顺序天然序列化，且调用方可以同步等待“写入完成/刷盘完成”的结果。

6.4.1 关键代码段：enqueue_and_wait（同步等待写入结果）

Listing 24: 入队并等待 writer 返回 Status（节选）

```
auto done = std::make_shared<std::promise<Status>>();
auto fut = done->get_future();
op.done = done;
{
    std::lock_guard<std::mutex> lk(queue_mu_);
    queue_.push_back(std::move(op));
}
queue_cv_.notify_one();
return fut.get();
```

该模式把 writer 线程内部错误（例如 fwrite 失败）可靠地传播到调用方，避免“写入失败但上层以为成功”。

6.5 刷盘策略（durability）

flush_durable 支持不同 sync 模式：

- **Off**: 仅 fflush;
- **Normal**: 按时间间隔节流 fsync/_commit;

在 Windows 上使用 _commit(_fileno(fp))，在类 Unix 上使用 fsync(fileno(fp))。

6.5.1 关键代码段：Normal 模式的节流刷盘

Listing 25: flush_durable: Normal 模式的时间间隔判断（节选）

```
if (sync_mode_ == WalSyncMode::Normal) {
    uint64_t now_ms = monotonic_ms();
    if (last_durable_flush_ms_ != 0 &&
        now_ms - last_durable_flush_ms_ < normal_sync_interval_ms_) {
        return Status::Ok();
    }
    last_durable_flush_ms_ = now_ms;
}
```

这是一种“吞吐优先”策略：在高频写入下减少 fsync/_commit 次数，但会扩大崩溃丢失窗口（取决于 interval）。

6.6 分段 (segment rotate)

当 WAL 文件大小超过阈值时，自动滚动到新段：

- 基础段为 db.wal
- 后续段为 db.wal.000001、db.wal.000002 ...

阈值通过环境变量 NEWDB_WAL_SEGMENT_BYTES 设置。

6.7 恢复 (recovery)

恢复逻辑的关键点：

- 可为每个段建立稀疏 offset 索引（每 128 条采样）以支持按最小 LSN seek；
- 按事务聚合：同一事务内同一 row_id 的多次写覆盖保存最后一次；
- COMMIT 时统一 apply 到 HeapTable，ROLLBACK 则丢弃；
- 最终 rebuild_indexes(schema) 以获得查询索引。

6.7.1 关键代码段：按事务聚合，commit 时统一 apply

Listing 26: recover: PendingTxnOps + COMMIT apply (节选)

```
// 事务内对同一 row_id 的操作，保留最后一次
PendingTxnOps& pending = txn_ops[hdr.txn_id];
pending.by_row_id[row.id] = { op, std::move(row) };

// COMMIT: 对该 txn 的所有最终行操作执行 apply
for (auto& [row_id, op_row] : it->second.by_row_id) {
    WalOp op = op_row.first;
    Row& row = op_row.second;
    if (op == INSERT || op == UPDATE) { upsert_into_table(row); }
    if (op == DELETE) { append_tombstone(row.id); }
}
txn_ops.erase(it);
```

该实现的含义是：**事务原子性通过“commit 时才生效”实现**；在 commit 之前读到 WAL 的中间状态也不会被 apply 到表中。

可用环境变量：

- NEWDB_RECOVER_ENABLE_OFFSET_SEEK=1
- NEWDB_RECOVER_MIN_LSN=<u64>

6.8 恢复阶段语义：analysis / redo / undo

在当前重构后的恢复链路中，WAL 不再只是“顺序回放”工具，而是承载更完整的事务恢复语义：

- **analysis**：扫描段文件并重建事务状态（活跃/提交/回滚边界），识别 checkpoint 与控制记录；
- **redo**：对已提交事务执行幂等重做，支持 barrier/strict 语义以避免重复 apply 造成数据漂移；

- **undo**: 对未完成事务执行回退, 保证 crash 后不会暴露未提交中间态。

这使恢复从“只做前滚”演进为“前滚 + 回滚”闭环, 语义上更接近成熟事务引擎的崩溃恢复模型。

6.9 PITR 与检查点协作

围绕恢复能力, 当前实现已支持按 LSN 或时间戳进行恢复边界控制 (PITR):

- **recoverToLsn**: 把恢复终点限制在目标 LSN;
- **recoverToTime**: 把恢复终点限制在目标时间点;
- **CHECKPOINT_BEGIN/END + PITR_MARK**: 用于缩短恢复扫描窗口并标记可回放边界。

工程收益是可将“恢复”从单一灾难动作, 扩展为“可控回放工具”(例如压测回看、事故窗口定位、版本回归验证)。

7 Session / C API: 缓存加载、快照设置与输出约束

对应实现: newdb/engine/src/session/api/session.cpp、newdb/engine/src/api/c/c_api.cpp、newdb/engine/src/schema/schema_io.cpp、newdb/engine/src/error/error_format.cpp

7.1 关键代码段: Session 如何确保已装载 (ensure_loaded)

Listing 27: Session::ensure_loaded 的核心逻辑 (节选)

```
if (s.data_path.empty()) {
    return Status::Fail("no table selected");
}
if (s.cache_valid) {
    return Status::Ok();
}
return session_reload_impl(s);
```

含义:

- **强约束: 必须先选表**: data_path 为空会直接失败; 上层应先设置 table/data 目录。
- **cache_valid 是核心开关**: 一旦 cache 有效, 后续读路径不会重复扫描 heap 文件。

7.2 关键代码段: reload 时的 schema sidecar + lazy 开关

Listing 28: session_reload_impl: .attr + NEWDB_LAZY_HEAP (节选)

```
const std::string sidecar = schema_sidecar_path_for_data_file(s.data_path);
Status s1 = load_schema_text(sidecar, s.schema);
HeapLoadOptions load_opts{};
if (std::getenv("NEWDB_LAZY_HEAP") == "1") {
    load_opts.lazy_decode = true;
}
```

```
Status s2 = io::load_heap_file(s.data_path.c_str(), s.table_name, s.schema, s.table, load_opts);
s.cache_valid = s2.ok;
```

这里把“schema 与数据装载”绑定在一起，保证 HeapTable 的索引/排序比较使用的是最新 schema 语义。

7.3 Session: cache_valid 与按需重载

Session 持有：

- data_path: 当前表的数据文件路径 (.bin)
- schema: 当前表 schema (来自 .attr sidecar)
- table: HeapTable 缓存
- cache_valid: 是否已装载且可复用

装载逻辑为：

1. load_schema_text(.attr): sidecar 缺失也视为“合法”，调用方采用默认 schema；
2. 读取环境变量 NEWDB_LAZY_HEAP=1 决定是否启用懒解码；
3. io::load_heap_file(.bin): 构建 HeapTable；
4. 成功后置 cache_valid=true。

7.4 快照设置：影响索引与排序

set_snapshot(snapshot) 会把 snapshot 下发给 HeapTable，并立即 rebuild_indexes(schema)。这意味着快照切换会触发一次索引重建（时间与数据量相关），但能保证之后所有读路径都遵循该快照视图。

7.5 关键代码段：C API 通过读取日志 tail 获取输出

Listing 29: newdb_session_execute: tail 读取与错误码（节选）

```
auto before = file_size(ptr->log_path);
process_command_line(ptr->shell, command_line);
TailReadResult tail = read_file_tail(ptr->log_path, before);
out = tail.data;
if (output_indicates_business_error(out)) {
    ptr->last_error = NEWDB_ERR_EXECUTION_FAILED;
    rc = 0;
}
prepend_capi_error_line(out, ptr->last_error);
```

设计权衡：

- **优点：**C ABI 输出缓冲区无需承载复杂结构，上层得到“与 CLI 一致”的文本输出。
- **风险：**错误判定依赖文本特征 (output_indicates_business_error)，可能出现误判或漏判；日志 IO 出错会导致执行失败。

7.6 错误格式：结构化单行输出

`format_error_line(domain, code, message)` 输出：

```
[ERR] domain=<domain> code=<code> message="<escaped>"
```

其中 `message` 会做必要的转义（反斜杠、双引号、换行/回车/制表符）。

7.7 C API：以日志尾部作为输出通道

`newdb_session_execute` 的输出策略是：

- 命令执行前记录 log 文件大小；
- 执行命令后读取 log 的 tail（新增部分）作为输出；
- 若输出包含特征性错误文本，则返回业务失败码；
- 最终把 `[CAPI_ERROR]` ... 前缀插入输出开头（便于上层解析）。

该设计的优点是：C ABI 无需传递复杂结构即可获得“近似完整”的命令输出；缺点是对日志 IO 有依赖，且错误判定依赖文本特征（需要注意误判/漏判）。

8 中层解析 I: cli (CLI、命令分发、业务编排)

8.1 模块定位

`cli/` 是“用户入口层 + 中层业务编排层”：

- **入口**：`cli/app/demo_main.cc` 解析 CLI 并决定执行模式；
- **命令分发**：`cli/shell/dispatch/router/dispatch.cc`、`cli/shell/dispatch/router/dispatch_routing.cc`（phase-2 前缀快路径）与 `cli/shell/dispatch/handlers/*` 负责交互命令解析与路由；
- **查询中间层**：`cli/modules/where/executor/*` 管理条件解析、候选集构建、策略限流与查询缓存；
- **事务协调**：`cli/modules/txn/coordinator/*` 封装 Txn + WAL + 文件锁 + vacuum 触发；
- **运行编排**：`cli/shell/bootstrap/demo_runner.cc` 对接 `batch/exec/import/mdb` 等模式。

8.2 关键数据结构（中层）

8.2.1 ShellState（会话上下文聚合）

Listing 30: ShellState（节选）

```
struct ShellState {
    newdb::Session session;
    std::optional<newdb::Session::HeapAccess> session_heap_guard;
    TxnCoordinator txn;
    std::string log_file_path{"demo_log.bin"};
    std::string data_dir;
```

```
bool encrypt_log{false};
bool verbose{false};
WhereQueryContext where_ctx;
};
```

说明:

- session_heap_guard 是核心: 在单条命令执行期内持有 Session::mut_, 保证缓存表指针有效;
- where_ctx 保存查询缓存/LRU/策略状态/统计, 避免每次 WHERE 都“冷启动”。

8.2.2 DemoCliInvocation (一次 CLI 调用的完整语义)

Listing 31: DemoCliInvocation (节选)

```
struct DemoCliInvocation {
    DemoCliWorkspace ws;
    bool encrypt_log{false};
    bool verbose{false};
    DemoPrimaryAction primary{CliInteractive{}};
    std::string error;
};
```

它把“参数解析结果”标准化为 workspace + flags + exactly one primary action, 后续 runner 可按 variant 分发。

8.2.3 WhereQueryContext (查询缓存与策略状态)

Listing 32: WhereQueryContext (节选)

```
struct WhereQueryContext {
    std::unordered_map<std::string, std::vector<std::size_t>> query_cache;
    std::list<std::string> query_cache_lru;
    std::atomic<std::uint64_t> cache_hits{0};
    std::atomic<std::uint64_t> policy_rejects{0};
    WherePolicyState policy{};
    mutable std::mutex mu;
};
```

其作用是把“WHERE 的性能状态”与“策略限流状态”显式挂在会话层, 而不是分散在静态全局变量中。

8.3 核心流程

8.3.1 A. 入口主流程 (demo_main)

1. 解析 CLI: demo_parse_invocation(argc, argv, inv)
2. 初始化运行上下文: demo_init_session_logging(app, ...)
3. 执行终端 phase (exec/batch/import/mdb): demo_try_run_terminal_phase(...)
4. 若无提前结束, 进入交互 shell。

8.3.2 B. 命令处理主链 (process_command_line)

Listing 33: 命令分发主链 (节选)

```
bool process_command_line(ShellState& st, const char* input_line) {
    // trim + logging + session commands
    if (handle_txn_commands(...)) return true;
    if (handle_workspace_admin_commands(...)) return true;
    if (handle_import_defattr_commands(...)) return true;
    if (handle_schema_catalog_commands(...)) return true;
    if (handle_ddl_create_use_commands(...)) return true;
    if (handle_ddl_alter_rename_commands(...)) return true;
    if (handle_schema_show_commands(...)) return true;
    if (handle_dml_insert_command(...)) return true;
    // fallback to cached HeapTable path
}
```

这条链的特点是“先高层语义命令、后表级缓存路径”，减少不必要的 heap 装载与解码。

8.4 事务与回退能力 (重构后现状)

围绕事务路径，cli/modules/txn/coordinator/* 与命令分发层已形成“可执行 + 可恢复 + 可观测”的闭环：

- **事务语法统一**：SAVEPOINT / ROLLBACK TO / RELEASE SAVEPOINT 在会话命令入口统一解释，降低命令分叉与 unknown 噪声；
- **任意回退语义**：支持按 savepoint 粒度回退与重做，命令层可组合为多步事务修复流；
- **崩溃恢复联动**：事务协调器与 WAL 的 analysis/redo/undo 恢复流程对齐，避免“命令层成功但恢复层不一致”；
- **诊断输出增强**：事务失败路径保留 soft/hard fail 诊断信息，便于自动化 gate 与 GUI 回看定位。

8.5 算法与复杂度 (中层视角)

- **CLI 分发**：参数扫描主要是 $O(\text{argc})$ ；
- **命令路由**：按 handler 顺序匹配，最坏近似 $O(H)$ (H =handler 数)；
- **WHERE**:
 - id/索引命中接近 $O(1)$ 或 $O(\log n)$ (取决于底层结构)；
 - fallback 全扫描路径为 $O(n)$ ；
 - 缓存命中后可将重复查询候选构建降到接近 $O(1)$ (不计输出)。

8.6 本章复评结论 (现状 / 风险 / 建议)

- **handler 链增长风险**：process_command_line 已较长，建议后续拆成“命令注册表 + 表驱动分发”。
- **会话锁持有时长**：session_heap_guard 设计正确，但需警惕重 I/O 命令在持锁期间阻塞。
- **策略可观测性**：可把 WhereQueryContext 的原子计数接入统一诊断命令，形成运行期画像。

8.7 近期修复要点: PAGE id 排序语义一致性

在 `cli/modules/sidecar/page/page_index_sidecar.cc` 的近期修复中, `PAGE(..., id, asc)` 明确改为按整数 id 比较, 避免字符串字典序导致的 1,10,2 问题 (表现为新增到 10 后显示在 1 与 2 之间)。

- **修复前:** id 在 sidecar 构建阶段以字符串键进入排序, 可能出现 $10 < 2$ 的错误顺序。
- **修复后:** 在排序收敛阶段对 `order_key=="id"` 走数值比较 ($a.id < b.id$ / $a.id > b.id$)。
- **兼容策略:** page sidecar header 引入版本字段 (ver), 旧索引自动失效重建, 避免历史缓存继续污染分页顺序。

这类问题本质上是“存储键编码语义与业务排序语义不一致”, 后续新增索引 sidecar 时应优先遵循:

排序比较规则必须与 `TableSchema::compare_attr` / 字段类型语义保持一致。

9 中层解析 II: engine/include (公共 API 与契约层)

9.1 模块定位

`newdb/engine/include/newdb/*.h` 是工程的“公共契约层”:

- 向 `cli/`、`tests/`、`tools/` 暴露稳定接口;
- 隐藏 `newdb/engine/src/` 的实现细节 (例如文件布局、线程模型、平台差异);
- 通过注释与类型设计明确线程边界、生命周期与错误传播方式。

9.2 头文件分层 (建议理解顺序)

1. 基础类型层: `error.h`、`row.h`、`schema.h`
2. 存储抽象层: `heap_page.h`、`heap_storage.h`、`heap_file_read_view.h`
3. 表与会话层: `heap_table.h`、`session.h`、`catalog.h`
4. 日志与并发层: `wal_manager.h`、`mvcc.h`、`session_history.h`
5. 编解码与 sidecar 层: `tuple_codec.h`、`wal_codec.h`、`schema_io.h`
6. 外部 ABI 层: `c_api.h`

9.3 关键契约示例

9.3.1 Status: 统一错误传播约定

Listing 34: `error.h` (节选)

```
struct Status {
    bool ok{true};
    std::string message;
    static Status Ok();
```

```
static Status Fail(std::string msg);
explicit operator bool() const { return ok; }
};
```

工程层面价值：减少异常跨模块传播的不确定性，形成一致的 `if (!st.ok) return st;` 风格。

9.3.2 Session：并发边界写在接口注释里

Listing 35: session.h (节选)

```
struct Session {
    class HeapAccess { ... }; // scoped lock holder
    HeapTable table;
    bool cache_valid{false};
    mutable std::mutex mut_;
    [[nodiscard]] HeapAccess lock_heap(const char* log_file);
    HeapTable* mutable_heap(const char* log_file);
};
```

`lock_heap` 与 `mutable_heap` 的并存体现了“安全优先路径”与“便捷路径”分层，调用方需明确线程模型。

9.3.3 WAL 管理接口：功能完整但实现可替换

Listing 36: wal_manager.h (节选)

```
class WalManager {
public:
    Status open();
    Status append_record(...);
    Status flush();
    Status recover(HeapTable* out_table, const TableSchema& schema, WalRecoveryStats* stats);
    std::optional<WalRecoveryStats> last_recovery_stats() const;
};
```

这层接口把“可观测恢复”变成一等能力（不仅仅是 `recover` 成功/失败二值）。

9.4 本章复评结论（现状 / 风险 / 建议）

- 优点：接口清晰，注释含线程/生命周期语义，便于跨目录协作；
- 不足：部分头文件体积较大（例如 `wal_manager.h`），后续可拆“配置/统计/事务包装”子头；
- 建议：增加一份“include 层稳定性等级”文档（稳定 ABI/内部 API/实验 API）降低演进风险。

10 中层解析 III: tests (验证策略与质量护栏)

10.1 测试分层概览

tests/ 基本可分为四层：

- 单模块语义测试：如 `test_schema.cpp`、`test_tuple_codec.cpp`、`test_error_format.cpp`

- 存储链路测试: 如 `test_page_io.cpp`、`test_heap_table.cpp`、`test_mvcc_visibility.cpp`
- 日志/并发/恢复测试: 如 `test_wal_concurrency.cpp`、`test_wal_segment.cpp`、`test_wal_recovery_indexed.cpp`
- cli/C API 集成测试: 如 `test_demo_where.cpp`、`test_c_api.cpp`、`test_demo_mdb_stop.cpp`、`test_dispatch_routing.cpp`

10.2 代表性测试样例

10.2.1 A. WHERE 语义与策略 (`test_demo_where.cpp`)

Listing 37: WHERE 多条件与策略拒绝 (节选)

```
std::vector<WhereCond> conds;
conds.push_back({"balance", CondOp::Ge, "15", ""});
conds.push_back({"balance", CondOp::Le, "25", "AND"});
const auto idx = query_with_index(tbl, schema, conds);

_putenv_s("NEWDB_WHERE_POLICY_MODE", "reject");
const auto blocked = query_with_index(tbl, schema, conds);
EXPECT_TRUE(blocked.empty());
EXPECT_TRUE(where_policy_last_blocked());
```

价值: 不仅验证“结果是否正确”, 还验证“策略路径是否按预期触发”。

10.2.2 B. 页 IO 容错与懒加载 (`test_page_io.cpp`)

Listing 38: 坏页跳过 + lazy decode 物化 (节选)

```
bad[idx] ^= 0x5A;
ASSERT_FALSE(newdb::heap_page::verify_checksum(bad.data(), bad.size()));
ASSERT_TRUE(newdb::io::load_heap_file(path.c_str(), "mix", schema, tbl).ok);

newdb::HeapLoadOptions opts; opts.lazy_decode = true;
ASSERT_TRUE(newdb::io::load_heap_file(path.c_str(), "lazy", schema, tbl, opts).ok);
ASSERT_TRUE(tbl.is_heap_storage_backed());
ASSERT_TRUE(tbl.materialize_all_rows(schema).ok);
```

价值: 覆盖“真实线上常见异常”(坏页/半页/缺文件)与“性能路径切换”(lazy→materialize)。

10.2.3 C. WAL 并发正确性 (`test_wal_concurrency.cpp`)

Listing 39: 多线程 append + commit (节选)

```
for (int t = 0; t < kThreads; ++t) {
    workers.emplace_back([&, t]() {
        for (int i = 0; i < kOpsPerThread; ++i) {
            wal.append_record(txn, newdb::WalOp::INSERT, "users", &r);
            wal.commit_transaction(txn);
        }
    });
}
```

```
wal.read_all_records(schema, records);
EXPECT_EQ(insert_count, kThreads * kOpsPerThread);
```

价值：验证 writer 队列与 LSN 递增在高并发下不会破坏可读性与计数一致性。

10.2.4 D. PAGE id 排序回归 (test_demo_cli.cpp)

Listing 40: PAGE id 升序最小回归场景 (节选)

```
newdb::HeapTable tbl;
tbl.rows = {
    newdb::Row{1, {"name", "a"}}, {},
    newdb::Row{10, {"name", "c"}}, {},
    newdb::Row{2, {"name", "b"}}, {},
};
PageSidecarRequest req{ /*order_key=*/"id", /*descending=*/false, ... };
const auto idx = load_or_build_page_index_sidecar(req, schema, tbl);
EXPECT_EQ(tbl.rows[idx[0]].id, 1);
EXPECT_EQ(tbl.rows[idx[1]].id, 2);
EXPECT_EQ(tbl.rows[idx[2]].id, 10);
```

价值：把“1,2,10 顺序正确性”固化为可重复验证的最小场景，避免后续 sidecar/排序重构时回归。

10.3 本章复评结论 (现状 / 风险 / 建议)

- 优点：

- 覆盖范围较广：语义、容错、并发、恢复、CLI/C API 都有触达；
- 场景设计贴近真实风险 (checksum 损坏、策略拒绝、并发压测)。

- 可优化点：

- 增加“属性/随机测试” (fuzz) 覆盖解析与编解码边界；
- 对性能基线测试设更清晰预算与波动容忍区间；
- 增加跨平台差异测试 (Windows/WSL/Linux 文件锁与路径语义)。

11 中层解析 IV: tools (基准与烟测工具链)

11.1 工具集定位

tools/ 的三个程序构成“性能-并发-冒烟”闭环：

- tools/perf/newdb_perf.cpp：调用 PowerShell 基准脚本，跑大规模场景；
- tools/perf/newdb_concurrent_perf.cpp：直接压 WAL 并发写入吞吐；
- tools/smoke/newdb_smoke.cpp：最小命令集 (create/load/append) 用于快速健康检查。

11.2 关键结构与流程

11.2.1 A. newdb_perf: 脚本编排器

Listing 41: newdb_perf 参数与执行 (节选)

```
struct PerfArgs {
    std::string demo_exe{"newdb_demo.exe"};
    std::string sizes_csv{"1000000"};
    int query_loops{1};
    int txn_per_mode{60};
};
cmd += "powershell -ExecutionPolicy Bypass -File ...million_scale_bench.ps1";
cmd += " -DemoExe " + quote_ps(args.demo_exe);
std::system(cmd.c_str());
```

它本质上是“参数整形 + 外部脚本调用”，将复杂基准逻辑留在脚本层，二进制工具只负责稳定入口。

11.2.2 B. newdb_concurrent_perf: 并发 WAL 压测

Listing 42: 并发写压测主循环 (节选)

```
for (int t = 0; t < args.threads; ++t) {
    threads.emplace_back([&, t, cnt]() {
        for (int i = 0; i < cnt; ++i) {
            wal.append_record(txn, newdb::WalOp::INSERT, "users", &r);
            wal.commit_transaction(txn);
            done.fetch_add(1);
        }
    });
}
wal.flush();
```

关键点:

- 默认 WalSyncMode::Off 评估“纯吞吐上限”;
- 可选 --verify 调 read_all_records() 校验 INSERT/COMMIT 计数一致性;
- 输出 tps/wal_bytes/elapsed_ms 便于 CI 横向比较。

11.2.3 C. newdb_smoke: 最小可用性验证

Listing 43: newdb_smoke 主命令 (节选)

```
if (cmd == "load") se.reload();
if (cmd == "create") newdb::io::create_heap_file(...), save_schema_text(...);
if (cmd == "append") newdb::io::append_row(...);
```

它用于开发早期与部署后“快速确认引擎活着且链路通”的场景。

11.3 本章复评结论 (现状 / 风险 / 建议)

- 优点:
 - 工具职责边界清晰, 便于脚本/CI 组合;
 - 并发压测工具直接命中 WAL 热路径, 定位性能回归很有效。
- 建议:
 - 为三工具统一输出 JSON 模式 (便于自动采集);
 - 把 sync mode、segment 大小、verify 粒度统一成可复用参数约定;
 - 增加跨平台 runner (目前 newdb_perf.cpp 偏 Windows PowerShell)。

12 中层解析 V: rust_gui (前端 GUI 独立文档与复评)

12.1 文档定位与范围

本章节将 newdb/rust_gui/ 作为**独立模块**评审, 覆盖:

- 前端渲染层: src/App.vue, src/styles.css
- Tauri 后端桥接层: src-tauri/src/lib.rs
- 运行时资源同步: scripts/sync_runtime_binaries.ps1

目标是回答三件事: GUI 架构是否清晰、可配置性是否完整、近期修复是否形成闭环。

12.2 架构分层复盘

12.2.1 A. 交互与状态层 (Vue)

- App.vue 承担主状态机: 表格页、MDB 编辑器、日志窗、CLI 窗、事务栈、设置弹窗。
- 关键状态分组已较完整: 数据态 (table/page/sort)、运行态 (busy/runtime dashboard)、UI 态 (modal/折叠/侧栏/主题)。
- 日志由“整段文本”升级为“逐行分类渲染”, 支持 cmd/error/success/meta/session 五类高亮。

12.2.2 B. 原生桥接层 (Tauri / Rust)

- lib.rs 暴露命令式接口: 执行命令、分页查询、runtime dashboard 读取、settings 持久化、CLI 终端管理。
- UiSettings 已使用 serde(default), 保证旧配置升级时缺字段不崩溃。
- 二进制/资源路径解析覆盖 dev 与 bundle 双场景, 容错能力优于早期版本。

12.2.3 C. 资源与交付层 (脚本)

- sync_runtime_binaries.ps1 统一同步 demo/perf/runtime_report/DLL 与脚本、results 种子文件。
- GUI 在离线打包后可直接读取 trend/dashboard 所需资源, 减少“本地有文件但 GUI 读不到”的环境差异。

12.3 近期关键变更归档 (本轮)

- **日志可观测性**: 新增会话分隔头 ([SESSION] ...) 并设为专属高亮类型。
- **回看效率**: 回看窗口支持 “命中统计 + 上/下条跳转 + 自动定位命中行”。
- **布局能力**: 侧栏支持显示/隐藏、宽度拖拽并持久化 (sidebarWidth)。
- **设置体系升级**:
 - 从单页堆叠改为 “弹窗侧栏菜单”;
 - 支持预设、导入导出 JSON、保存状态反馈;
 - 扩展到圆角/阴影/日志排版/边框对比度/面板亮度/日志高亮强度。
- **数据顺序修复**: PAGE 的 id 排序改为数值语义, 并补最小回归测试 (1,2,10)。

12.4 事务栈能力 (Undo/Redo)

Rust GUI 当前已不只是 “命令面板”, 而是具备事务历史编排能力的操作层:

- **栈模型升级**: 以 savepoint 级 UndoUnit 聚合命令明细 UndoOp, 支持跨表事务回退;
- **执行策略**: 后端区分 soft/hard fail, 失败时按策略修复或中止, 避免半回退状态静默扩散;
- **持久化恢复**: 事务栈以 jsonl 持久化, 启动时可恢复历史并对坏行容错;
- **逆向推断增强**: 针对 UPDATE/DELETE/SETATTR 等操作优先走结构化查询快照推断 inverse, 降低文本解析误差;
- **可观测性**: 回看面板支持命中导航、会话分段、逆向来源展示, 便于审计与复盘。

12.5 运维与验证闭环

GUI 与脚本层已形成面向 CI/soak 的闭环:

- **脚本镜像**: sync_runtime_binaries.ps1 递归镜像 newdb/scripts 到 GUI 运行目录, 确保 gate/bench/soak 脚本一致;
- **看板联动**: crash matrix 与压力结果可输出 JSON 并进入 nightly trend/dashboard;
- **设置持久化**: 面板、表格视图、日志视图透明度等关键 UI 参数可持久化恢复, 降低重启后状态漂移。

12.6 本章复评结论 (现状 / 风险 / 建议)

12.6.1 结论 1: 结构可维护性明显提升

- 设置系统从 “零散硬编码” 走向 “变量驱动 + 持久化配置”, 维护成本下降。
- 侧栏菜单化后, 设置弹窗的信息密度与可达性更稳定, 后续新增配置项更可控。

12.6.2 结论 2: 用户感知问题闭环较完整

- 已闭环的问题: 日志混会话、回看定位慢、侧栏操作割裂、分页 id 错序。
- 已形成 “修复 + 回归测试 + GUI 同步” 的链路, 降低同类问题复发概率。

12.6.3 结论 3：仍有可优化空间（非阻塞）

- **组件拆分**：App.vue 仍偏大，建议逐步拆成 SettingsPanel / LogPanel / SidebarPanel。
- **主题深度**：当前以强度参数为主，下一步可增加模块级基色（侧栏/主区/控制台独立色板）。
- **日志性能**：日志量继续增长时，可引入虚拟列表减少 DOM 压力。

12.7 建议的后续演进顺序

1. 先拆组件边界（降低 App.vue 复杂度）；
2. 再补主题第三轮（模块基色 + 对比度自适应）；
3. 最后做日志虚拟化（在高日志量下保证交互流畅）。

该顺序兼顾“工程可维护性”和“用户可感知收益”，能避免一次性大改导致回归面过大。

A 附录：与 *LevelDB* / *InnoDB* 底层结构对比

本附录从“存储组织、索引结构、日志与恢复、并发控制、空间管理、读写放大”六个维度，对比 newdb、*LevelDB*、*InnoDB* 的底层结构差异。目标不是简单判断优劣，而是帮助定位：当前 newdb 的工程形态更接近哪些设计取舍，以及后续演进时可借鉴的方向。

A.1 一图总览（结构定位）

- **newdb**：页式 heap 文件 + append 风格记录版本 + 可选懒加载 + WAL + 基础 MVCC 快照过滤。
- **LevelDB**：LSM-Tree（MemTable + Immutable MemTable + SSTable 多层合并）+ WAL。
- **InnoDB**：B+Tree 聚簇存储（Clustered Index）+ 二级索引 + Redo/Undo + Buffer Pool + 完整事务系统。

A.2 1) 存储组织（Data Layout）

A.2.1 newdb

- 主数据是 .bin heap 文件，按固定页（Waterfall page）组织。
- 页内用 header + payload + slot 表管理记录，不为每条记录显式存长度（通过 offset 差分恢复）。
- 同一 row.id 的新版本以追加方式写入，装载时合并为“最新视图”。

A.2.2 LevelDB

- 写入先进入 MemTable（跳表），随后刷成 SSTable（不可变有序文件）。
- 多层 SSTable 通过 compaction 合并，形成典型 LSM 分层结构。
- 逻辑删除依赖 tombstone 与 compaction 清理，不是“原地删除”。

A.2.3 InnoDB

- 以页为单位管理表空间，主数据组织在聚簇索引 B+Tree 中。
- 数据行按主键顺序存放；二级索引叶子保存主键值回表。
- Buffer Pool 承担页缓存与脏页管理，刷盘由后台线程调度。

A.3 2) 索引与查询路径

- **newdb**: 内存态 hash 索引 (`index_by_id` / `index_by_pk_value`) + 排序缓存；懒加载时索引值指向 slot，需要二次解码。
- **LevelDB**: 查询路径是 MemTable + Immutable + 多层 SSTable (Bloom Filter + 索引块 + 数据块)；范围查询友好，点查性能受层数与 compaction 状态影响。
- **InnoDB**: B+Tree 原生支持点查/范围查；有自适应哈希索引等优化，整体查询语义最完整。

A.4 3) WAL / Redo 与恢复模型

A.4.1 newdb

- WAL 记录头 (LSN/txn/type/checksum) + payload (table + row fields)。
- writer 线程串行写入，支持 sync mode 与 segment rotate。
- 恢复时按事务聚合，COMMIT 才 apply；提供 recovery stats 便于诊断。

A.4.2 LevelDB

- WAL 主要保障 MemTable 崩溃恢复；重启时先重放 WAL，再恢复到可服务状态。
- 核心一致性更多依赖 LSM 不可变文件与 manifest 元数据维护。

A.4.3 InnoDB

- Redo Log (物理/生理日志) 保证持久性；Undo Log 支持回滚与 MVCC 可见性。
- Checkpoint、Doublewrite、Crash Recovery 机制成熟，恢复语义最完整。

A.5 4) 并发控制与 MVCC 深度

- **newdb**: 有 MVCCSnapshot 与可见性判断 (`created/deleted/active_txns`)，并已具备 savepoint 级回退与 analysis/redo/undo 恢复闭环；但在锁管理与高隔离级别语义上仍弱于 InnoDB。
- **LevelDB**: 以单机 KV 引擎语义为主，事务与复杂隔离级别不是其核心目标。
- **InnoDB**: 完整事务隔离 (RC/RR 等)、行锁/间隙锁、undo 版本链，MVCC 与锁协作成熟。

A.6 5) 空间回收与长期运行形态

- **newdb**: append + tombstone 模式下，历史版本可能膨胀；当前主要靠装载合并与逻辑过滤。
- **LevelDB**: 依赖 compaction 回收冗余版本与 tombstone，是系统常态机制。
- **InnoDB**: 页分裂/合并、purge、后台线程协同管理，空间回收策略复杂但长期稳定。

A.7 6) 写放大 / 读放大 (工程后果)

- **newdb:**
 - 写放大较低 (追加写友好);
 - 若缺少周期性 compact, 读路径可能因历史版本/解码变慢。
- **LevelDB:**
 - 写放大来自 compaction;
 - 通过顺序写与批量合并换取高吞吐。
- **InnoDB:**
 - 写路径复杂 (redo + page flush + 二级索引维护);
 - 换来更强事务语义与更均衡的通用查询能力。

A.8 7) 对 newdb 的演进启发 (可落地)

- **借鉴 LevelDB:** 补 “后台 compact/merge” 机制, 减少 tombstone 与历史版本堆积。
- **借鉴 InnoDB:** 若目标走 OLTP, 需补全 undo 版本链、锁管理与更强恢复语义。
- **保持 newdb 特色:** 继续强化懒加载 + 页级顺序访问 + 轻量 WAL 的可观测性 (stats), 适配教学/实验/中小负载场景。

A.9 8) 结论 (结构层面)

- newdb 当前更像 “**页式存储 + LSM/OLTP 中间态**”: 已有 WAL、快照、页布局、懒加载、savepoint 回退与 PITR, 但仍未形成 InnoDB 级锁/隔离体系, 也未引入 LevelDB 式系统性 compaction。
- 若追求简单可控与可读性, 现结构非常适合继续迭代; 若追求长期高并发生产语义, 需要在 “版本回收 + 事务子系统 + 后台调度” 上补关键能力。

A.10 9) 优劣评价 (工程决策视角)

A.10.1 newdb

优势

- 结构清晰、可读性高: 页结构、WAL、快照可见性链路短, 便于教学与快速实验。
- 追加写友好: 直接 append 到 heap 页, 写路径相对轻量。
- 可观测性不错: WAL 恢复统计 (stats) 和显式状态返回有助于排障。

劣势

- 缺少系统化后台 compact/purge, 长期运行可能出现历史版本与 tombstone 堆积。
- 锁管理与隔离级别深度不足, 距离完整 OLTP 引擎仍有明显差距。
- 索引体系偏 “内存态辅助索引”, 对超大规模数据与复杂查询支持有限。

A.10.2 LevelDB

优势

- LSM 结构对写入吞吐非常友好，顺序写特性突出。
- 实现成熟、工程经验丰富，嵌入式 KV 场景稳定可靠。
- Bloom Filter + 分层 SSTable 设计，在典型 KV 负载下性价比高。

劣势

- compaction 带来的写放大与尾延迟需要精细调参。
- 原生事务/隔离能力弱，不适合直接承担复杂关系型事务语义。
- 范围查询与点查性能会受层级状态和 compaction 节奏影响。

A.10.3 InnoDB

优势

- 事务、MVCC、锁管理、恢复体系完整，适合通用 OLTP 生产场景。
- B+Tree 结构对点查和范围查询都较均衡，SQL 生态完备。
- 长期运行治理能力强（purge/checkpoint/buffer pool 等机制成熟）。

劣势

- 架构复杂，学习和调优成本高。
- 写路径与后台机制多，系统开销和维护门槛明显高于轻量引擎。
- 在纯 KV 高写吞吐场景下，不一定优于专门的 LSM 引擎。

A.10.4 按场景选型建议

- 教学/实验/原型验证：优先 newdb（可控、可读、易改造）。
- 单机嵌入式 KV 高写吞吐：优先 LevelDB（LSM 体系成熟）。
- 通用事务型业务（SQL + 强一致事务）：优先 InnoDB。

A.11 10) 纳入中层（cli/engine-include/tests/tools）后的再评估

前文 1)~9) 主要基于底层结构。将中层模块纳入后，newdb 的工程画像需要做一次修正：

A.11.1 A. 架构完整性：从“内核原型”提升为“可用小系统”

- newdb/cli/ 提供了从 CLI 到命令分发、事务协调、查询策略的完整入口链路；
- newdb/engine/include/ 提供公共契约层（Status/Session/WAL/C API），使模块边界更清晰；
- tools/ 补齐了基准与冒烟工具，tests/ 补齐了回归与并发护栏。

结论：newdb 不再只是“底层存储实验”，而是具备了可验证、可演示、可持续迭代的最小数据库工程闭环。

A.11.2 B. 与 *LevelDB*/*InnoDB* 的差距重述（中层视角）

- **相对 *LevelDB***: newdb 在“可读性、可教学、可改造”上优势明显，且已补齐 analysis/redo/undo + PITR 恢复链路；但在大规模长期运行治理（compaction 体系化、尾延迟控制）仍弱。
- **相对 *InnoDB***: newdb 在架构透明度与学习曲线方面更友好，且已具备 savepoint 级回退能力；但事务隔离深度、锁子系统、后台调度完整性仍有代际差距。

换言之，newdb 已从“可用性增强”进入“事务恢复能力显著增强”阶段，但底层成熟度仍与 *InnoDB*/*LevelDB* 的长期生产能力存在代差。

A.11.3 C. 工程风险再分级（加入中层后）

- **已明显下降的风险**: 接口失配风险（engine/include 统一契约）、回归风险（tests 覆盖）、使用门槛风险（cli/tooling）、崩溃后事务不一致风险（redo/undo 恢复闭环）。
- **仍高的核心风险**: 版本回收与空间膨胀、复杂并发语义（锁/隔离）、长期稳定性与运维自动化。

A.11.4 D. 选型建议修订（纳入中层能力）

- **newdb 适用边界上移**: 从“课程/实验”上移到“教学 + 原型 + 中小规模内部工具型负载（可控场景）”。
- **仍不建议直接替代**: 在高并发强事务生产核心链路上，不建议直接替代 *InnoDB*；在超大规模嵌入式 KV 连续写场景，不建议直接替代成熟 LSM（如 *LevelDB*/*RocksDB*）。
- **最佳定位**: 作为“可解释数据库内核 + 工程化脚手架”平台最有价值，适合做算法验证、课程实验、定制化功能孵化。

A.11.5 E. 下一阶段优先级（基于中层现状）

1. 建立后台 compact/purge 主线，降低历史版本与 tombstone 积累；
2. 在事务子系统补齐冲突检测与更明确隔离语义；
3. 将 tools/tests 与性能预算绑定，形成持续性能回归基线。

A.12 Executive Summary（1 页决策速览，面向评审会）

一句话结论

- newdb 目前已从“可读的底层原型”升级为“可演示、可验证、可迭代的小型数据库系统”，并在事务恢复上形成 analysis/redo/undo + savepoint + PITR 能力闭环；但底层成熟度仍处于 *LevelDB*/*InnoDB* 之下，短期定位应是“教学/原型/内部可控负载”，而非生产核心替代。

当前定位（纳入中层后）

- **技术形态**: heap 页存储 + WAL + analysis/redo/undo 恢复 + savepoint 回退 + PITR + 中层 CLI/契约/测试/工具闭环。
- **工程能力**: 可快速迭代、可回归验证、可做性能观测（含并发压测与恢复统计）。
- **核心短板**: 版本回收体系、锁与隔离深度、长期运行治理能力。

与对标系统的决策差异

- **对 LevelDB**: newdb 更易解释和改造；LevelDB 在大规模写入与长期 compaction 治理更成熟。
- **对 InnoDB**: newdb 学习曲线更低；InnoDB 在事务、锁、恢复、运维体系上显著领先。
- **决策含义**: newdb 的竞争力来自“可控性/可解释性/可定制性”，不是“极限成熟度”。

建议选型边界

- **推荐**: 课程教学、算法实验、功能原型、内部中小规模工具型场景（可控 SLA）。
- **谨慎**: 中高并发且对隔离语义敏感的业务链路。
- **不建议直接替代**: 生产核心 OLTP（InnoDB 目标域）与超大规模高写 KV 主战场（LevelDB/RocksDB 目标域）。

未来 2 个里程碑（建议）

1. **M1: 稳定性里程碑**（先可长期跑）
 - 上线 compact/purge 主线，控制版本堆积；
 - 固化性能预算（吞吐/延迟/恢复时间）并接入 CI 回归。
2. **M2: 事务能力里程碑**（再可承载更严肃业务）
 - 补齐事务冲突检测与隔离语义边界；
 - 强化并发一致性测试矩阵与故障注入恢复测试。

评审会可直接使用的决策建议

- **立项建议**: 继续投入，定位“数据库内核教学与原型孵化平台”。
- **资源建议**: 优先投向 compact/purge 与事务语义，不优先追求 SQL 面扩张。
- **验收建议**: 以“长期运行稳定性 + 回归可测性 + 性能基线达标”作为阶段性 Gate。

A.13 11) 成熟度对照（已补齐 / 待补齐）

A. 与早期版本相比已补齐的关键项

- **崩溃恢复闭环**: 由“仅重放”升级为 analysis/redo/undo 三阶段恢复。
- **回退语义**: 支持 savepoint 级回退/重做与事务级回滚。
- **时间点恢复**: 支持按 LSN/时间戳进行 PITR 边界控制。
- **可观测性与自动化**: crash matrix、恢复统计与 nightly 趋势链路可联动。

B. 相对 LevelDB/InnoDB 仍待补齐的关键项

- **LevelDB 方向**: 系统化 compaction 与长期尾延迟治理能力。
- **InnoDB 方向**: 锁管理与更完整隔离级别语义（不仅是可见性过滤与回退）。
- **通用方向**: 长期运行自动化治理（后台调度、容量回收、SLO 驱动告警）。